

算法设计与分析

Design & Analysis of Algorithms

第2章 递归与分治策略

学习要点

2.1 递归概念。

2.2 分治策略。

2.3 通过下面的范例学习分治策略设计技巧。

(1) 二分搜索技术；

(2) 大整数乘法；

(3) 棋盘覆盖；

(4) 快速排序

(5) 线性时间选择；

2.8 快速排序

```
template<class Type>
void QuickSort (Type a[], int p, int r)
{
    if (p<r) {
        int q=Partition(a, p, r);
        QuickSort (a, p, q-1); //对左半段排序
        QuickSort (a, q+1, r); //对右半段排序
    }
}
```

在快速排序中，记录的比较和交换是从两端向中间进行的，关键字较大的记录一次就能交换到后面单元，关键字较小的记录一次就能交换到前面单元，记录每次移动的距离较大，因而总的比较和移动次数较少。

2.8 快速排序

```
template<class Type>
int Partition (Type a[], int p, int r)
{
    int i = p, j = r + 1;
    Type x=a[p];// a[p] pivot枢轴
    // 将< x的元素交换到左边区域
    // 将> x的元素交换到右边区域
    while (true) {
        while (a[++i] < x);
        while (a[--j] > x);
        if (i >= j) break;
        Swap(a[i], a[j]);
    }
    a[p] = a[j];
    a[j] = x;
    return j;
}
```

{ 6, 7, 5, 2, 5, 8} 初始序列

{ 6, 7, 5, 2, 5, 8} j--;

{ 5, 7, 5, 2, 6, 8} i++;

{ 5, 6, 5, 2, 7, 8} j--;

{ 5, 2, 5, 6, 7, 8} i++;

{ 5, 2, 5} 6 {7, 8} 完成

2.8 快速排序

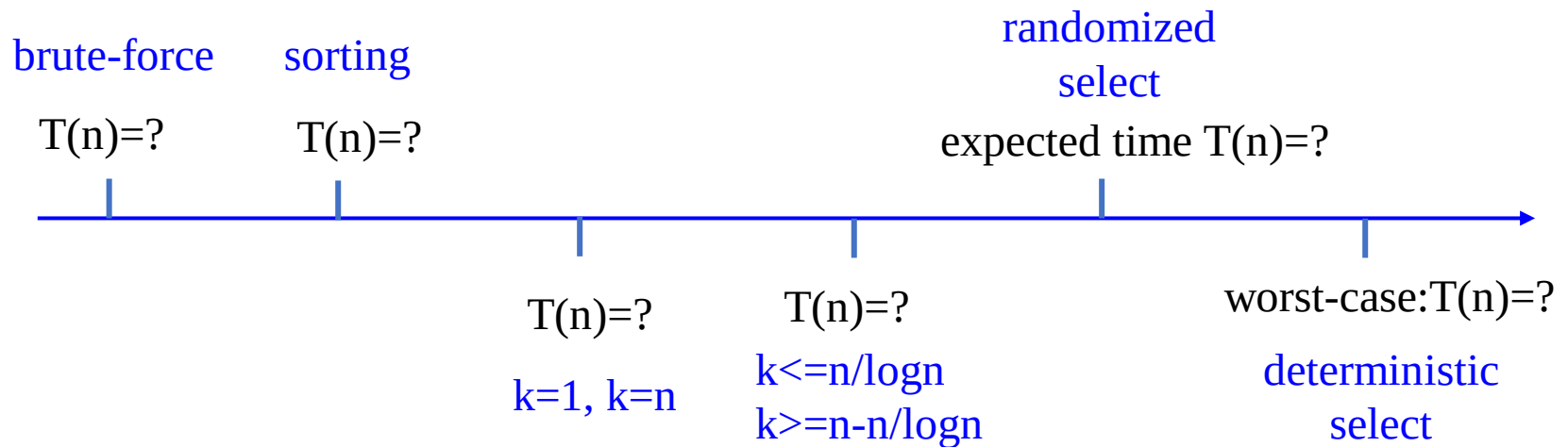
- 快速排序算法的性能取决于划分的对称性。
- 通过修改**partition**，可以优化快速排序算法
- 随机选择策略：当数组还没有被划分时，可以在 $a[p:r]$ 中随机选出一个元素作为划分基准，这样可以使划分基准的选择是随机的，从而可以期望划分是较对称的。

```
template<class Type>
int RandomizedPartition (Type a[], int p, int r)
{
    int i = Random(p,r);
    Swap(a[i], a[p]);
    return Partition (a, p, r);
}
```

- 最坏时间复杂度： $O(n^2)$ 随机化后： $1.38n\log n$
- 平均时间复杂度： $O(n\log n)$

2.9 线性时间选择

给定线性序集中 n 个元素和一个整数 k , $1 \leq k \leq n$, 要求找出这 n 个元素中第 k 小的元素。



2.9 线性时间选择

给定线性序集中 n 个元素和一个整数 k , $1 \leq k \leq n$, 要求找出这 n 个元素中第 k 小的元素。

```
template<class Type>
```

```
Type RandomizedSelect(Type a[], int p, int r, int k)
```

```
{
```

```
    if (p==r) return a[p];
```

```
    int s=RandomizedPartition(a, p, r),
```

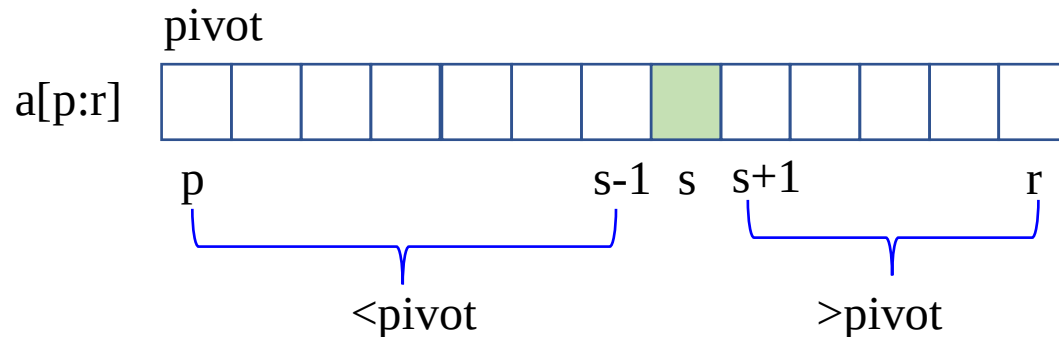
```
    j=s-p+1;
```

```
    if (k<=j) return RandomizedSelect(a, p, s, k);
```

```
    else return RandomizedSelect(a, s+1, r, k-j);
```

```
}
```

- 快排算法划分过程中, pivot 左侧都比pivot值小。
- 通过对划分的两部分计数可以找到第 k 小的元素。



在最坏情况下, 算法**randomizedSelect**需要 $O(n^2)$ 计算时间

算法**randomizedSelect**可以在 $O(n)$ 平均时间内找出 n 个输入元素中的第 k 小元素。

2.9 线性时间选择

如果能在线性时间内找到一个划分基准, 使得按这个基准所划分出的2个子数组的长度都至少为原数组长度的 ε 倍($0 < \varepsilon < 1$ 是某个正常数), 那么就可以在**最坏情况下**用 $O(n)$ 时间完成选择任务。

例如: 若 $\varepsilon=9/10$, 算法递归调用所产生的子数组的长度至少缩短 $1/10$ 。所以, 在最坏情况下, 算法所需的计算时间 $T(n)$ 满足递归式 $T(n) \leq T(9n/10) + O(n)$ 。由此可得 $T(n) = O(n)$ 。

Divide and Conquer:

base case: $n < n_0$, sorting

divide: $O(n)$ -MoM-partition

conquer: $T(s-p)$ or $T(r-s)$

combine: zero

Q1: how to find MoM?

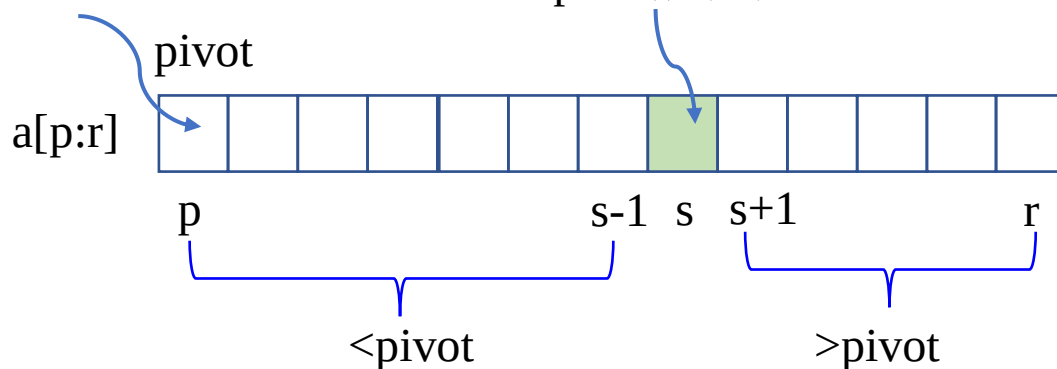
Q2: $\max\{|s-p|, |r-s|\} = ?$

Q3: $T(n) \leq O(n)$?

Median of Medians, MoM

中位数组中位数

pivot所在位置



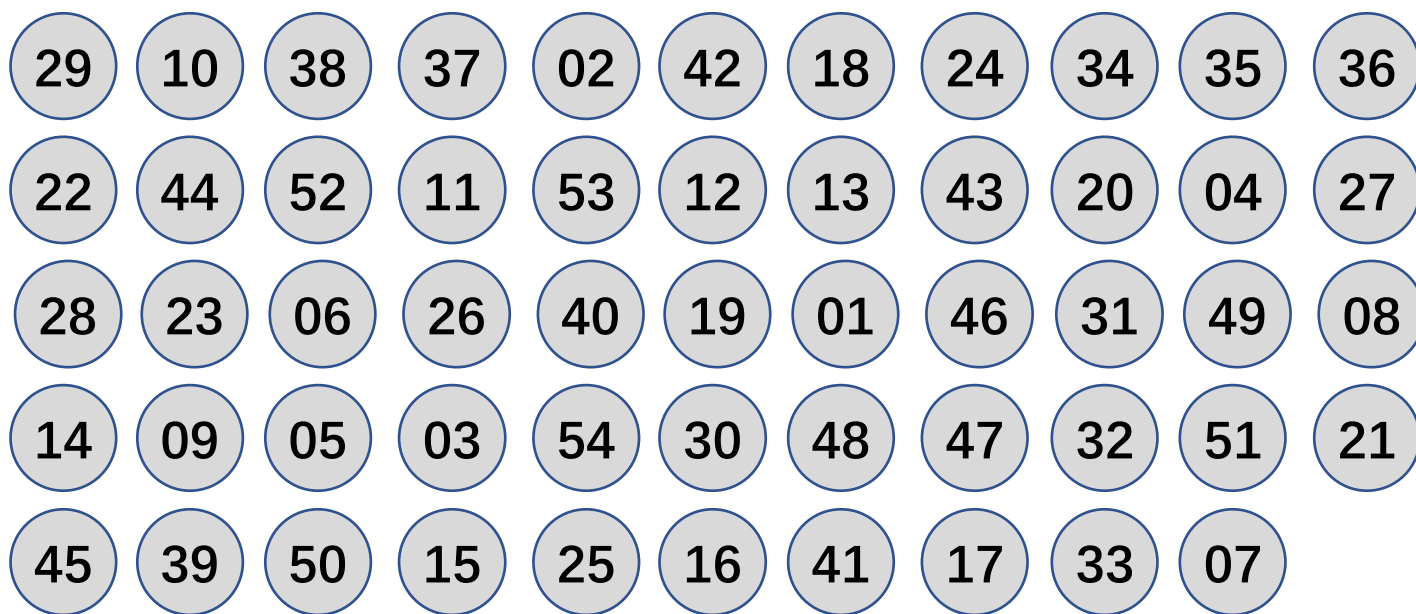
2.9 线性时间选择 实例

$n=54, k=15$

递归初始条件为 $n>5$

Median of Medians, MoM

中位数组中位数



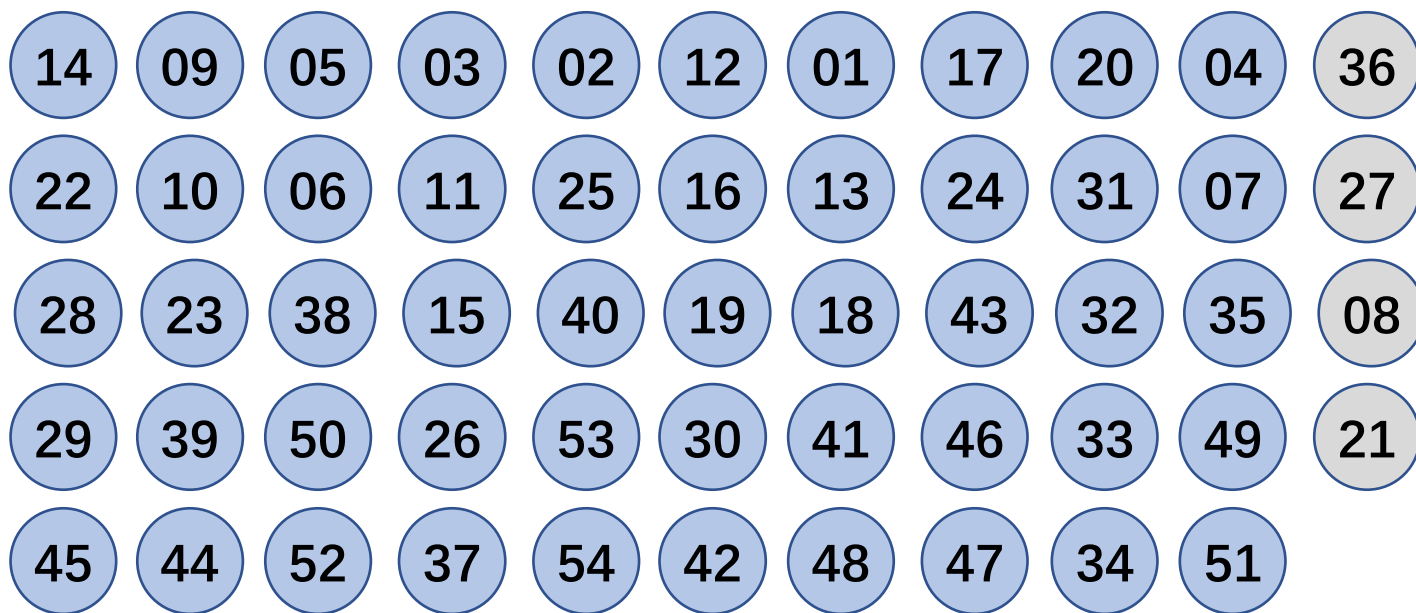
2.9 线性时间选择 实例

$n=54, k=15$

递归初始条件为 $n>5$

Median of Medians, MoM

中位数组中位数



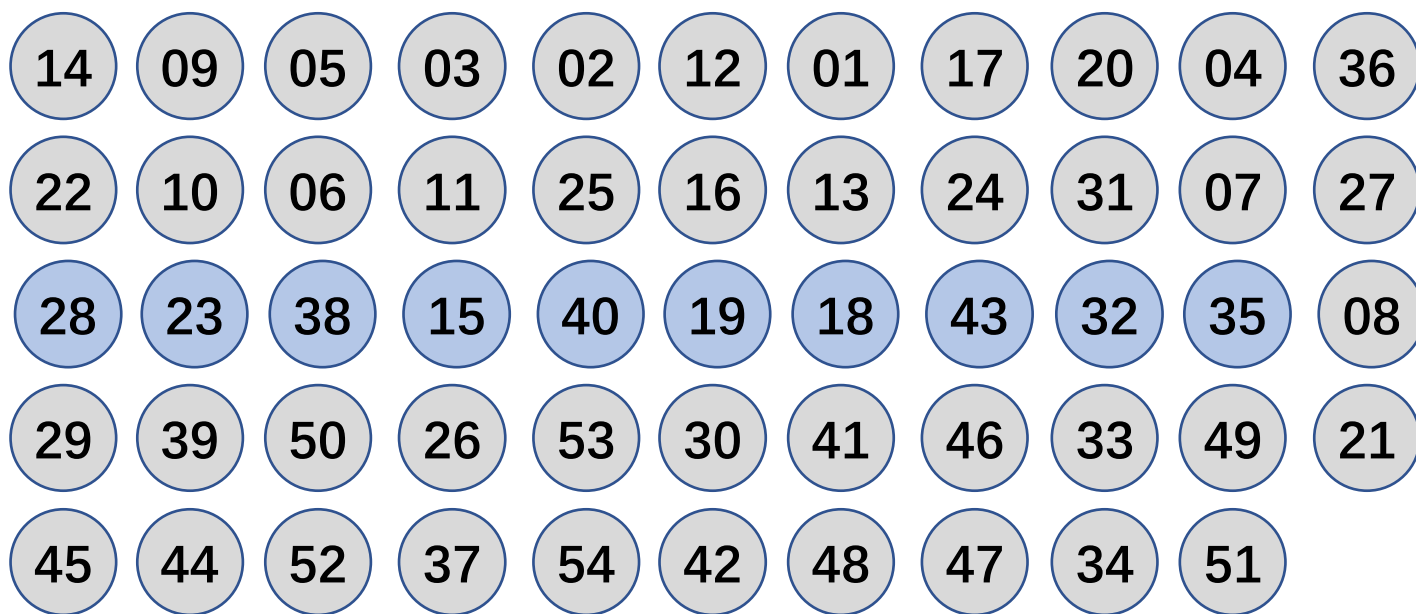
2.9 线性时间选择 实例

$n=54, k=15$

递归初始条件为 $n>5$

Median of Medians, MoM

中位数组中位数



取出每组中位数

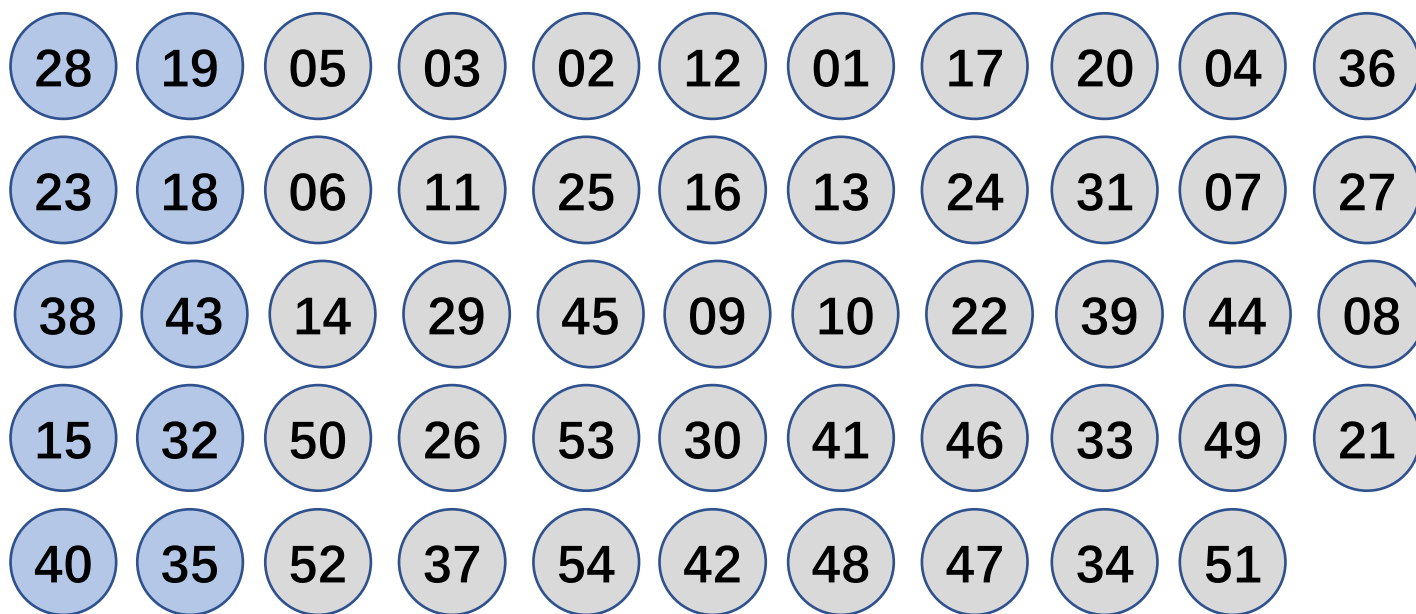
2.9 线性时间选择 实例

$n=54, k=15$

递归初始条件为 $n>5$

Median of Medians, MoM

中位数组中位数



构造中位数数组

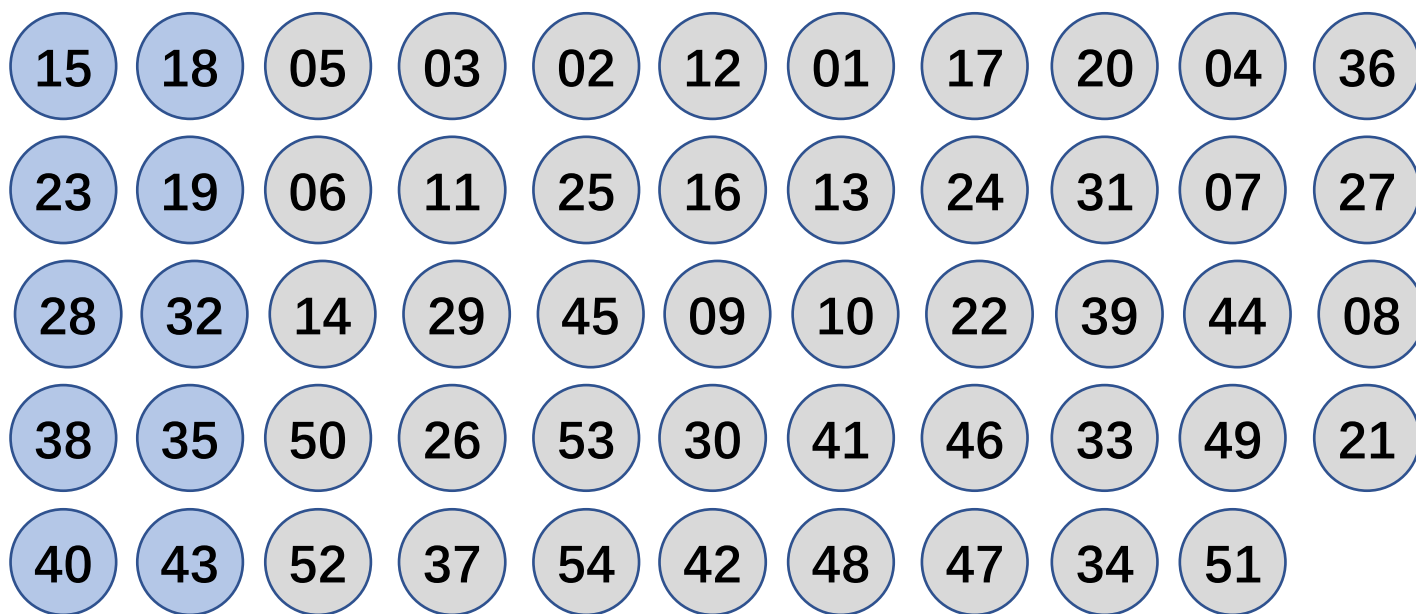
2.9 线性时间选择 实例

$n=54, k=15$

递归初始条件为 $n>5$

Median of Medians, MoM

中位数组中位数



每组 $O(1)$ 排序

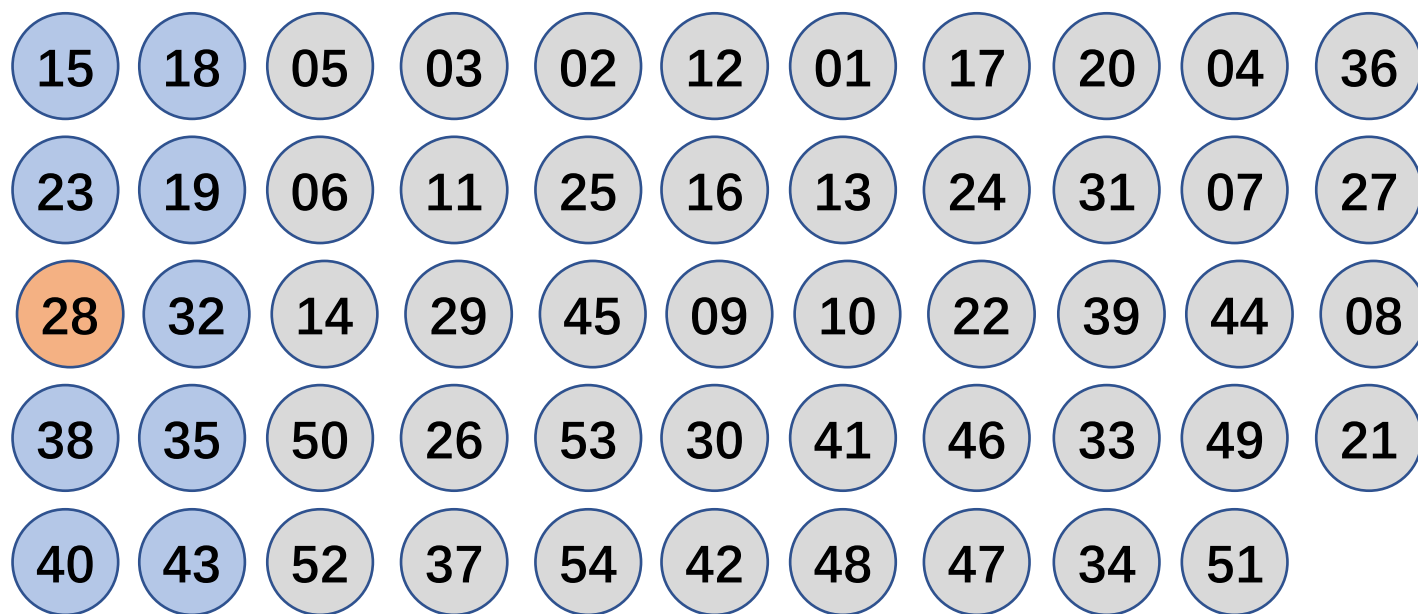
2.9 线性时间选择 实例

$n=54, k=15$

递归初始条件为 $n>5$

Median of Medians, MoM

中位数数组中位数



取出中位数数组[28,32]的中位数28作为
pivot对中位数数组[15,23,...,43]进行partition

2.9 线性时间选择 实例

$n=54, k=15$

递归初始条件为 $n>5$

Median of Medians, MoM

中位数数组中位数

pivot



取出中位数数组[28,32]的中位数28作为
pivot对中位数数组[15,23,...,43]进行partition

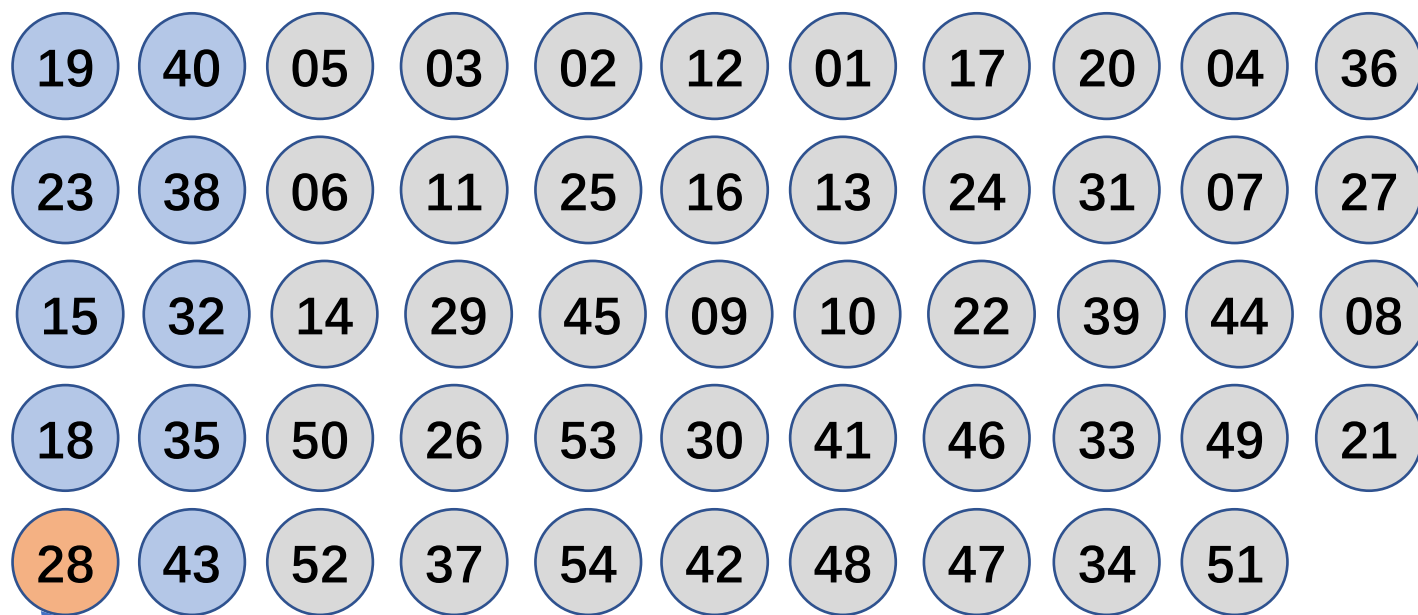
2.9 线性时间选择 实例

$n=54, k=15$

递归初始条件为 $n>5$

Median of Medians, MoM

中位数数组中位数



Median of Medians

取出中位数数组[28,32]的中位数28作为
pivot对中位数数组[15,23,...,43]进行partition

2.9 线性时间选择 实例

$n=54, k=15$

递归初始条件为 $n>5$

Median of Medians, MoM

中位数组中位数

pivot



将MoM作为pivot进行 $O(n)$ partition

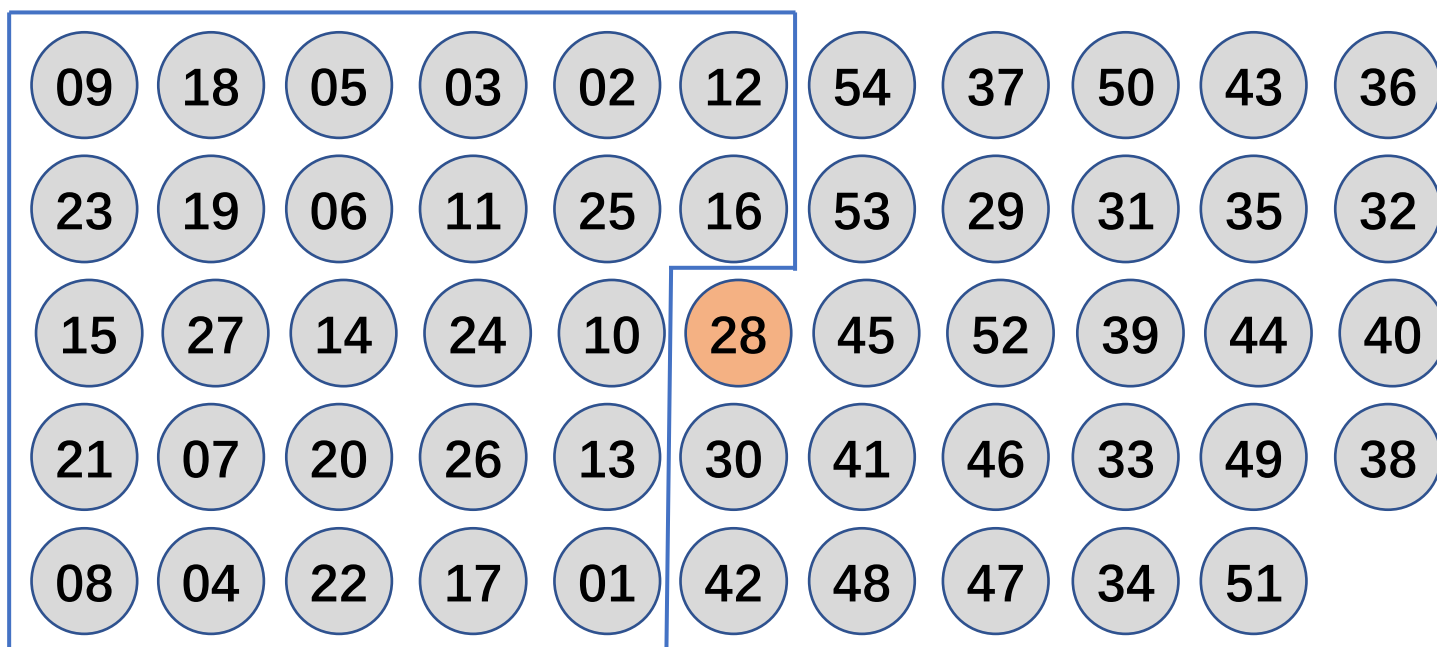
2.9 线性时间选择 实例

$n=54, k=15$

递归初始条件为 $n>5$

Median of Medians, MoM

中位数组中位数



一轮partition后pivot排好序

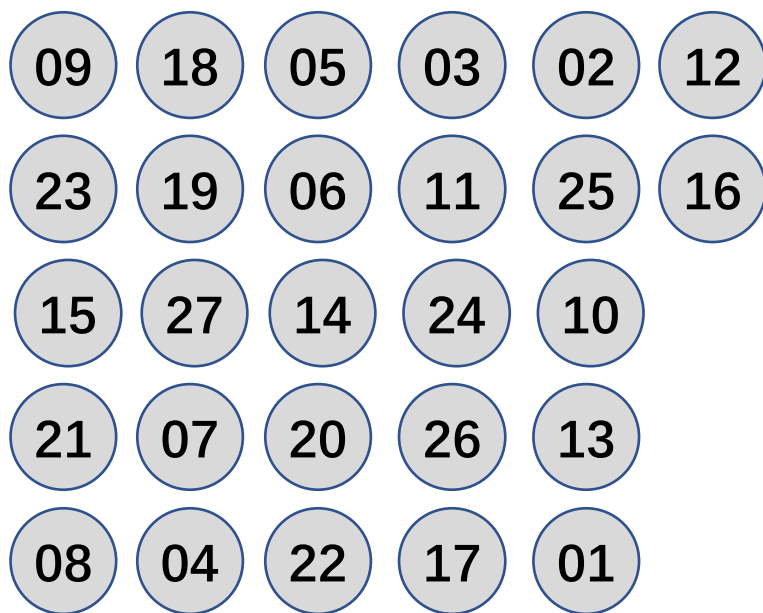
2.9 线性时间选择 实例

$n=54, k=15$

递归初始条件为 $n>5$

Median of Medians, MoM

中位数组中位数



$s > k$

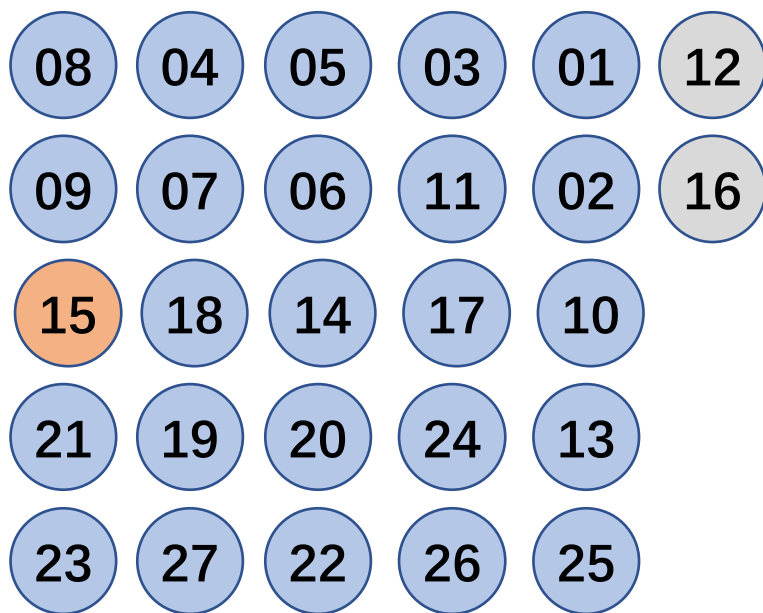
2.9 线性时间选择 实例

$n=54, k=15$

递归初始条件为 $n>5$

Median of Medians, MoM

中位数组中位数



2.9 线性时间选择 实例

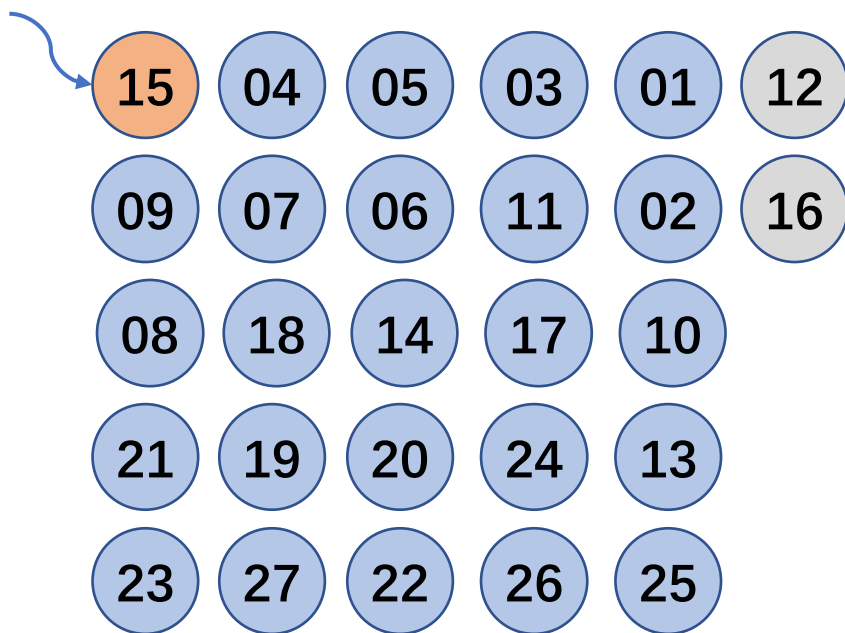
$n=54, k=15$

递归初始条件为 $n>5$

Median of Medians, MoM

中位数组中位数

pivot



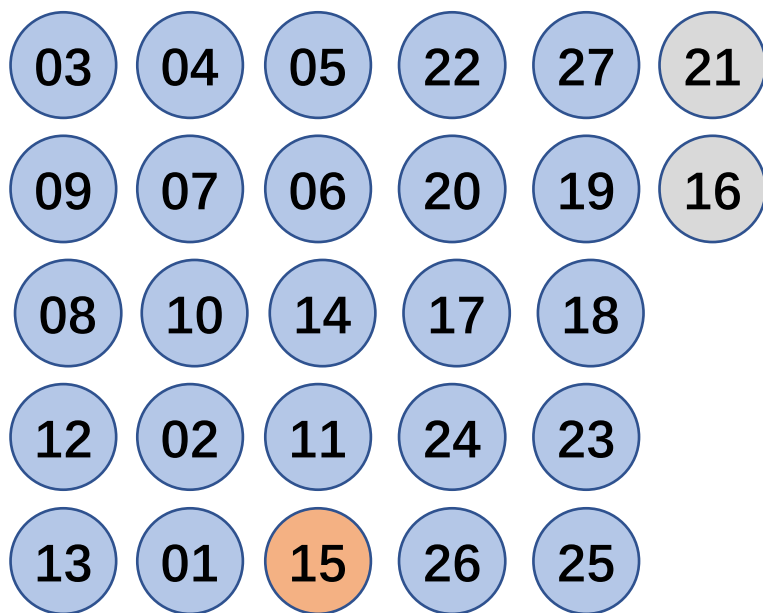
2.9 线性时间选择 实例

$n=54, k=15$

递归初始条件为 $n>5$

Median of Medians, MoM

中位数组中位数



2.9 线性时间选择

Q1:how to find MoM?

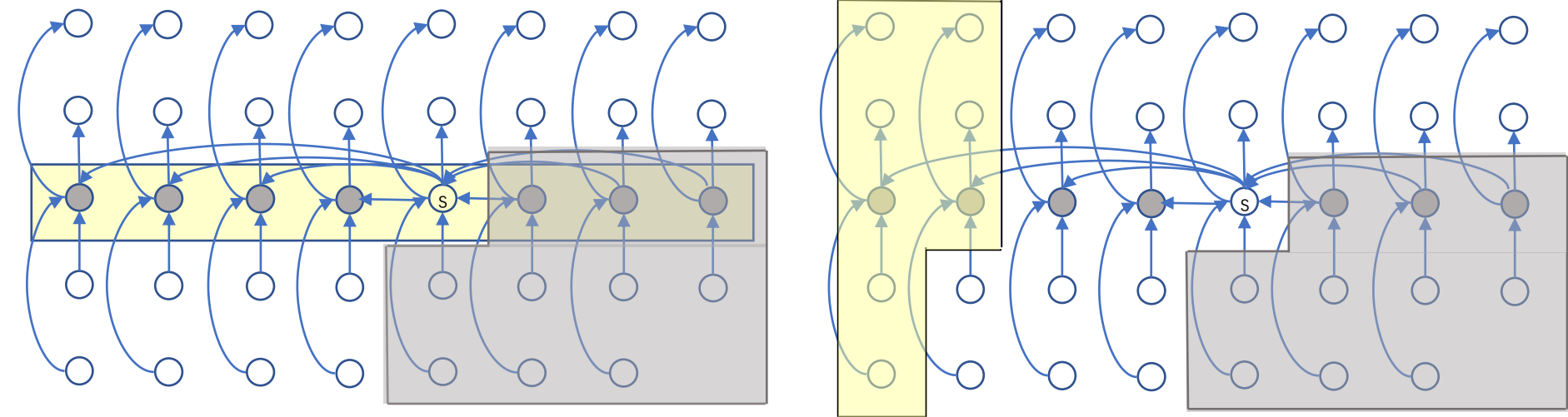
Select($a, p, (r-p-4)/5, (r-p-4)/10$) 每组5个

- base case: $n < n_0$, sorting
- divide: $\lceil n/5 \rceil$ medians
- conquer: $T(\lceil n/5 \rceil)$
- combine: zero

$a, p, (r-p-4)/5, (r-p-4)/10$

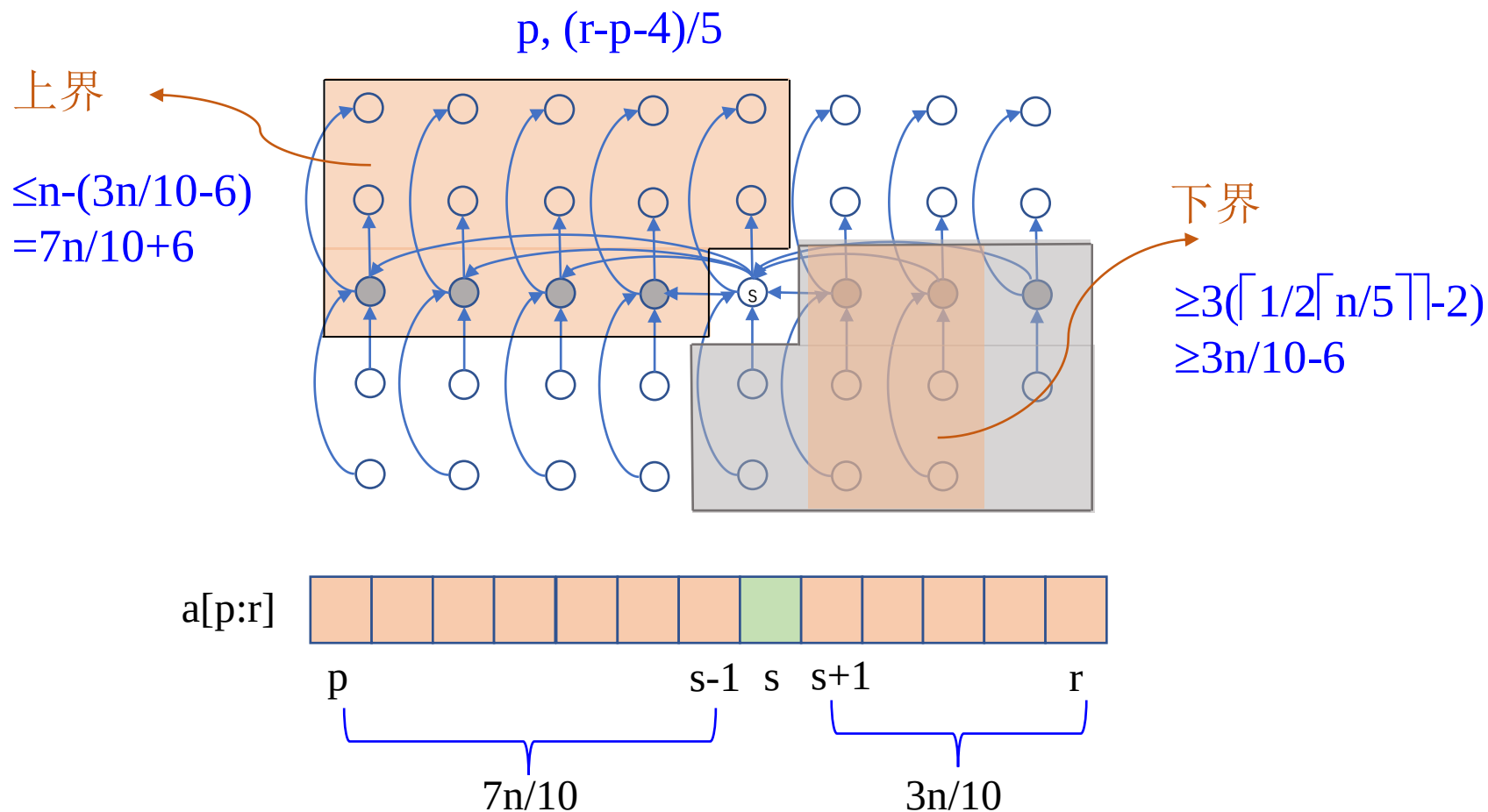
递归向下找中位数数组的中位数

$p, (r-p-4)/5$



2.9 线性时间选择

Q2: $\max\{|s-p|, |r-s|\}=?$



Q3: $T(n) \leq O(n)?$

$n \geq n_0, T(n) \leq T(n/5) + T(7n/10) + O(n)$

线性时间选择

Type **Select**(Type a[], int p, int r, int k)

```
{
    if (r-p<75) {
        用某个简单排序算法对数组a[p:r]排序;
        return a[p+k-1];
    };
    for ( int i = 0; i<=(r-p-4)/5; i++ )
        将a[p+5*i]至a[p+5*i+4]的第3小元素
        与a[p+i]交换位置;
    //找中位数的中位数， r-p-4即上面所说的n-5
    Type x = Select(a, p, p+(r-p-4)/5, (r-p-4)/10);
    int i=Partition(a,p,r, x),
    j=i-p+1;
    if (k<=j) return Select(a,p,i,k);
    else return Select(a,i+1,r,k-j);
}
```

上述算法将每一组的大小定为5，并选取75作为是否作递归调用的分界点。这2点保证了 $T(n)$ 的递归式中2个自变量之和 $n/5 + 3n/4 = 19n/20 = \varepsilon n$, $0 < \varepsilon < 1$ 。这是使 $T(n)=O(n)$ 的关键之处。当然，除了5和75之外，还有其他选择。

复杂度分析

$$T(n) \leq \begin{cases} C_1 & n < 75 \\ C_2 n + T(n/5) + T(3n/4) & n \geq 75 \end{cases}$$

$T(n)=O(n)$

实验2-5：线性时间选择

根据上页函数填空。

本章小结

- 例2.3 Fibonacci数列
- 例2.4 全排列问题
- 例2.5 整数划分问题
- 大整数乘法
- 棋盘覆盖
- 线性时间选择

上机总结要求：

1. 问题模型
2. 算法实例
3. 程序代码
4. 程序验证
5. 算法分析